

COMP2054-ACE: Introduction to big-Oh

ADE-Lec02b

Lecturer: Andrew Parkes

Email: andrew.parkes@Nottingham.ac.uk

from <http://www.cs.nott.ac.uk/~pszajp/>

Recap: Aim: **Classification of Functions**

- In computer science, we often need a way to group together **functions** by their scaling behaviour, and the classification should
 - Remove unnecessary details
 - Be (relatively) quick and easy
 - Be able to deal with 'weird' functions that can happen for runtimes
 - Still be mathematically well-defined
- Experience of CS is that this is best done by the "big-Oh notation and family"

Recap: Definition: “Big-Oh”

For two positive functions $f(n)$ and $g(n)$, of a single variable n , then

$f(n)$ is $O(g(n))$ “ f is Big-Oh of g ”

if and only if there exist constants $c \geq 0$ and $n_0 \geq 1$ such that

for all $n \geq n_0$. $f(n) \leq c g(n)$

RECAP EXERCISE

$f(n)$ is $O(g(n))$ iff $\exists c, n_0$ s.t.
 $\forall n \geq n_0. f(n) \leq c g(n)$

- It is easy to show directly from the definition that the function:

$$f(n) = 1 \text{ is } O(n)$$

I.e. we have both

1 is $O(n)$ - “strange but true”

1 is $O(1)$ - “natural”

Nothing in the definition forces a choice of a “best” or “most useful” $g(n)$.

EXERCISE (1)

- Consider the function:

$$f(n) = \begin{cases} n & \text{if } n \text{ is even} \\ 1 & \text{if } n \text{ is odd} \end{cases}$$

What is its big-Oh behaviour?

EXERCISE (2)

- Consider the function:

$$f(n) = \begin{cases} n & \text{if } n \text{ is even} \\ 1 & \text{if } n \text{ is odd} \end{cases}$$

What is its big-Oh behaviour?

ANS: It is $O(n)$.

Proof: if $n \geq 1$ then $f(n) \leq n$

hence can take $c=1$ $n_0=1$

EXERCISE (3)

- Consider the function:

$$f(n) = \begin{cases} n & \text{if } n \text{ is even} \\ 1 & \text{if } n \text{ is odd} \end{cases}$$

What is its big-Oh behaviour?

ANS: It is $O(n)$.

Proof: if $n \geq 1$ then $f(n) \leq n$

hence can take $c=1$ $n_0=1$

Note: despite the two cases, "even"/"odd", the definition requires to provide one c and one n_0 .

EXERCISE (4)

- Consider the function:

$$f(n) = \begin{cases} n & \text{if } n \text{ is even} \\ 1 & \text{if } n \text{ is odd} \end{cases}$$

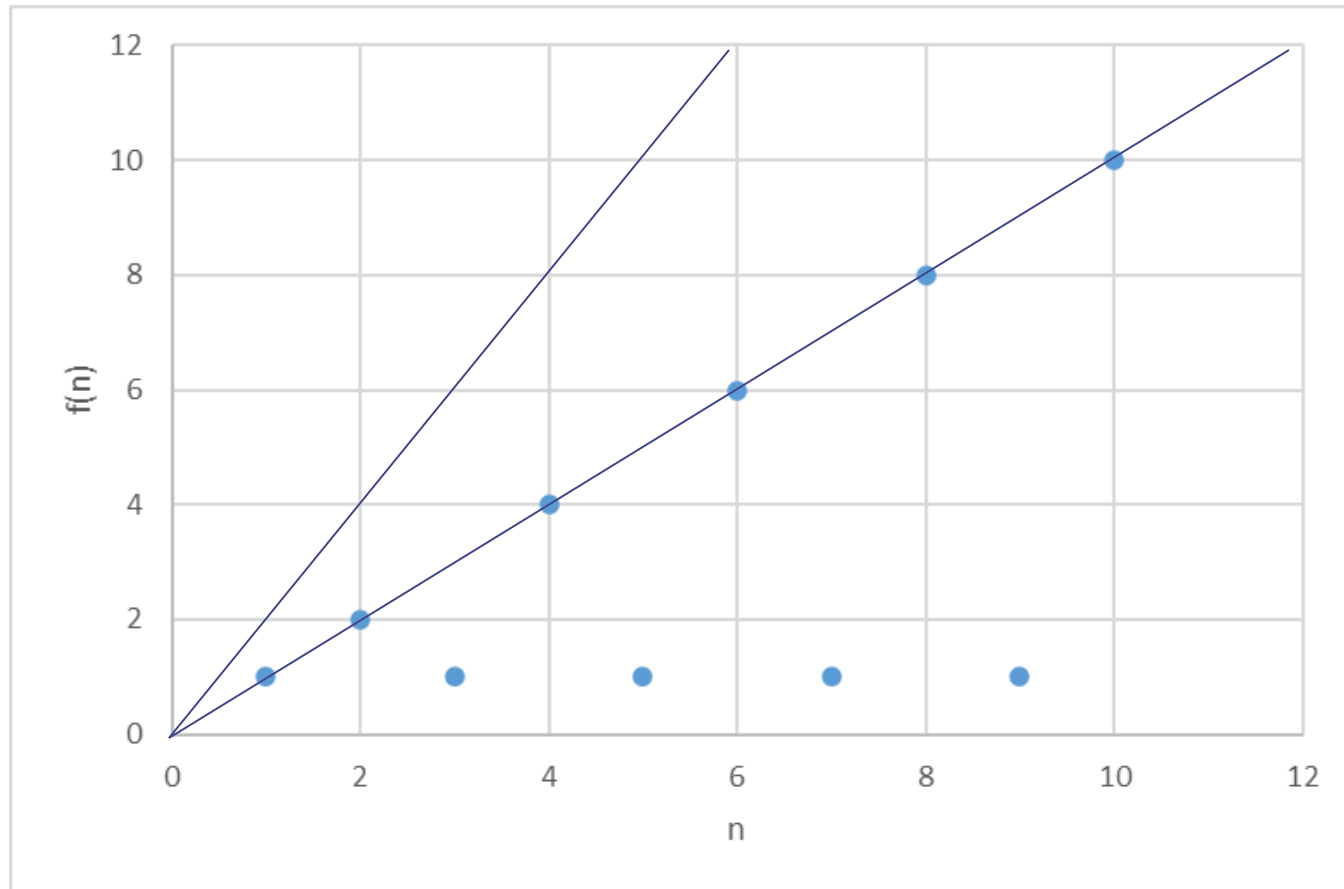
Is it $O(1)$?

ANS: No, as we would need

Exists c, n_0 . $n \leq c \cdot 1$ for all $n \geq n_0$

And this fails for the cases n is even.

Big-Oh Graphically



EXERCISE (5)

- Consider the function:

$$f(n) = \begin{cases} n & \text{if } n \text{ is even} \\ 1 & \text{if } n \text{ is odd} \end{cases}$$

What is the limit of the ratio $f(n)/n$ as n tends to ∞ (infinity, i.e. becomes very large)?

EXERCISE (ans)

- Consider the function:

$$f(n) = \begin{cases} n & \text{if } n \text{ is even} \\ 1 & \text{if } n \text{ is odd} \end{cases}$$

What is the limit of the ratio $f(n)/n$?

ANS: It does not have a limiting ratio !

$f(n)/n = 1$ if n is even, limit is 1

$1/n$ if n is odd, limit is 0

Shows how “big-Oh” handles weird functions and is different from limits.

Ratios vs. big-Oh

- $f(n)$ can be $O(g(n))$ even if the ratio $f(n)/g(n)$ does not exist
- Hence, big-Oh can be used in situations that ratios cannot
- The possibility of 'weird functions' means that big-Oh is more suitable than ratios for doing analysis of efficiency of programs

EXERCISE (6)

- Consider the function:

$$f_2(n) = \begin{cases} n & \text{if } n \text{ is even} \\ 4 & \text{if } n \text{ is odd} \end{cases}$$

What is its big-Oh behaviour?

EXERCISE (7)

- Consider the function:

$$f_2(n) = \begin{cases} n & \text{if } n \text{ is even} \\ 4 & \text{if } n \text{ is odd} \end{cases}$$

What is its big-Oh behaviour?

ANS: It is $O(n)$.

Proof: if $n \geq 4$ then $f_2(n) \leq n$

hence can take $c=1$ $n_0=4$

“other providers of c and n_0 are available”

EXERCISE

- Consider the function:

$$f_2(n) = \begin{cases} n & \text{if } n \text{ is even} \\ 4 & \text{if } n \text{ is odd} \end{cases}$$

What is its big-Oh behaviour?

ANS: It is $O(n)$.

Not a (full) proof “from the definition”:

for even case: pick $c=1$ $n_0=1$

for odd case: pick $c=4$ $n_0=1$.

(In lean: your proof would fail)

Big-Oh Usage

Consider the 2 functions:

$$f1(n) = n \quad \text{if } n \text{ is even}$$

$$1 \quad \text{if } n \text{ is odd}$$

$$f3(n) = 1 \quad \text{if } n \text{ is even}$$

$$n^2 \quad \text{if } n \text{ is odd}$$

Intuitively, $f1$ has much better scaling behaviour. However, we cannot get “ $f1$ is better” from simple comparison of values, because:

$$f1(n) > f3(n) \text{ if } n \text{ is even, but}$$

$$f1(n) < f3(n) \text{ if } n \text{ is odd}$$

However, we do have

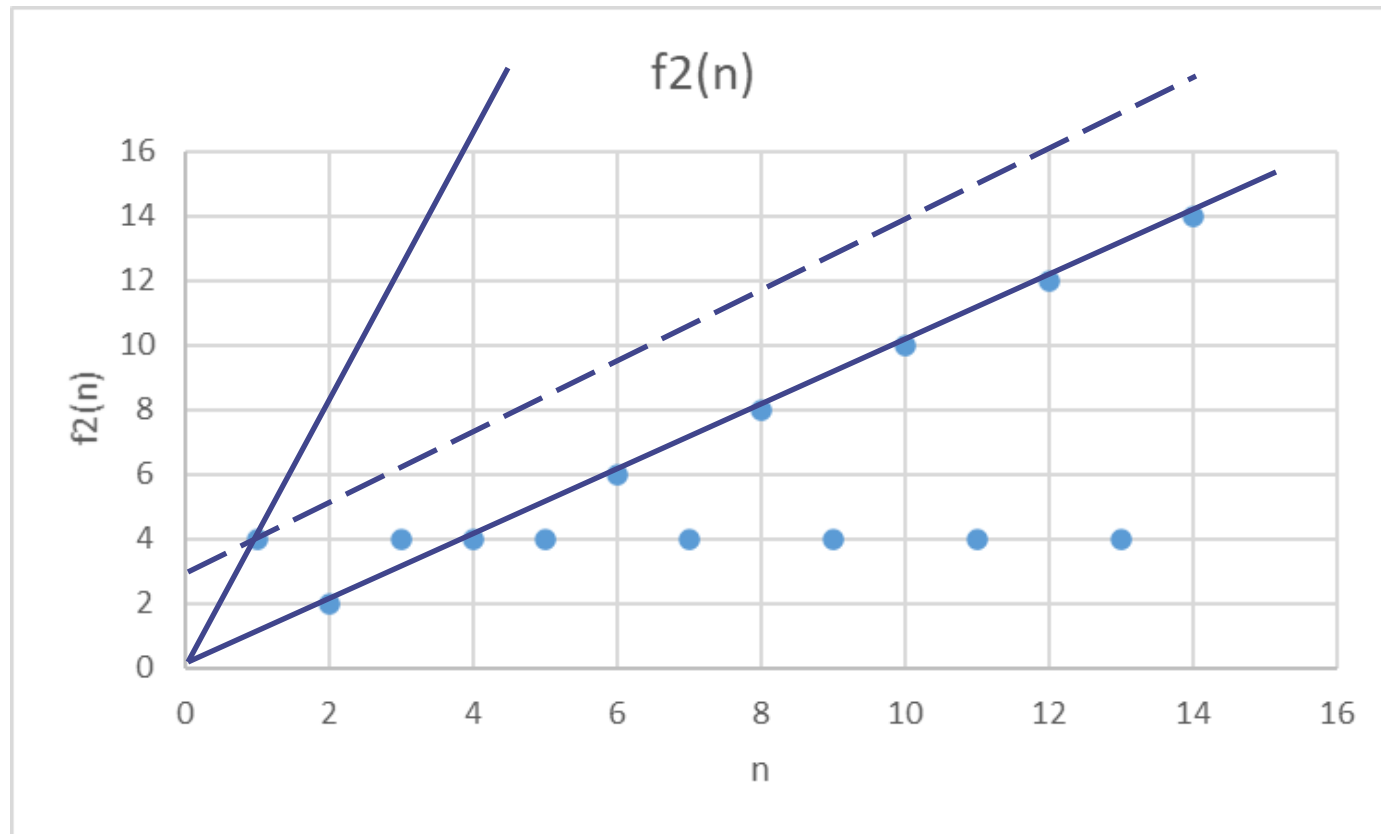
$$f1 \text{ is } O(n)$$

$$f3 \text{ is not } O(n), \text{ but only is } O(n^2)$$

Which “demonstrates $f1$ is better”

EXERCISE (9)

- For the $f2$ from slide 15:
- “Messy at small n ” ; fine past $n=4$



Exercises (offline)

- From the definition, show that
 - $(3n-6)$ is $O(4n+5)$
 - $(3n-6)$ is $O(n)$
 - $(4n+5)$ is $O(n)$
- Notes
 - The first shows that $g(n)$ does not need to be “nice”
 - The last two ‘suppress details’ as desired

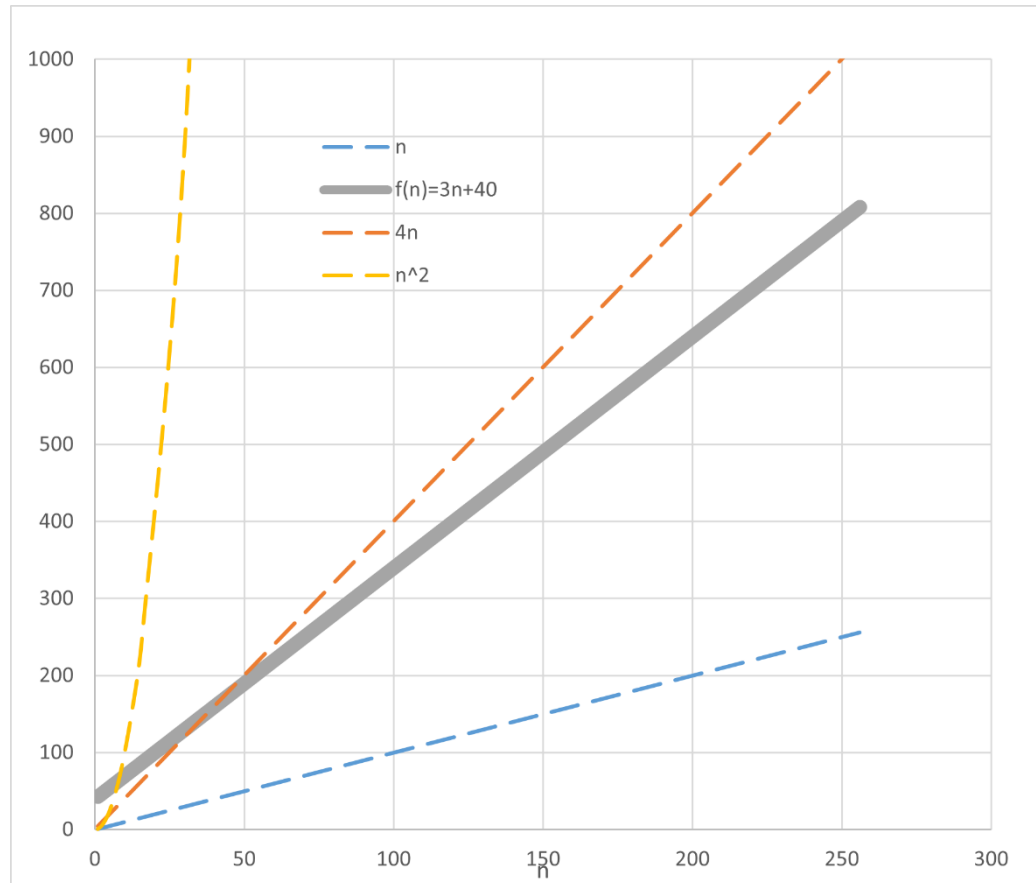
Comment on some details:

- $f(n) = (3n-6)$ is not always positive, but only is negative on a finite number of values of n
- Hence, to be more strict:
 - Could instead consider $f(n) = \max(0, 3n-6)$ is really meant
 - Or could just require that n_0 is taken large enough that $f(n)$ is positive: that is require
$$\forall n \geq n_0. \quad 0 \leq f(n) \leq c g(n)$$

Either of these capture the intent that “ f grows no faster than g at large enough n ”

Big-Oh in pictures: lin-lin

- Consider $f(n) = (3n+40)$
- It is 'obviously' linear, $O(n)$, e.g. take $c=4$
- Plot shows $f(n) \leq 4n$ eventually
- Also, shows it is $O(n^2)$



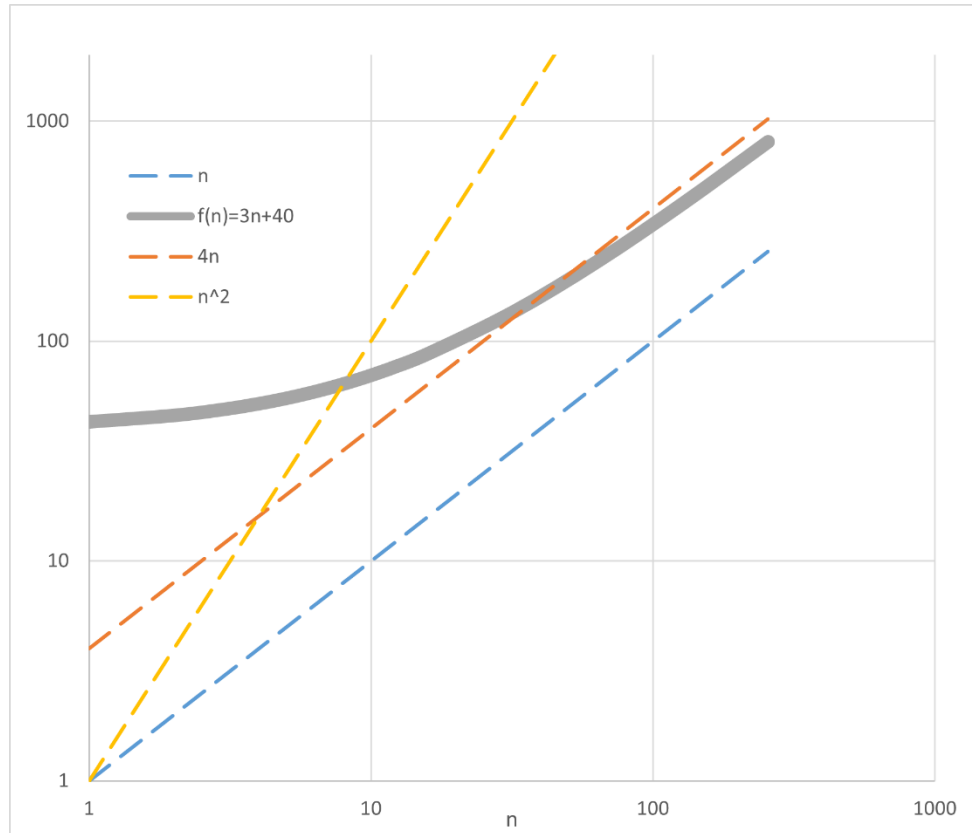
Big-Oh in pictures: log-log

The quadratic upper bound is clearer on a log-log scale

Use

$$\log(n^k) = k \log(n)$$

So higher powers have larger slope and will always win eventually.



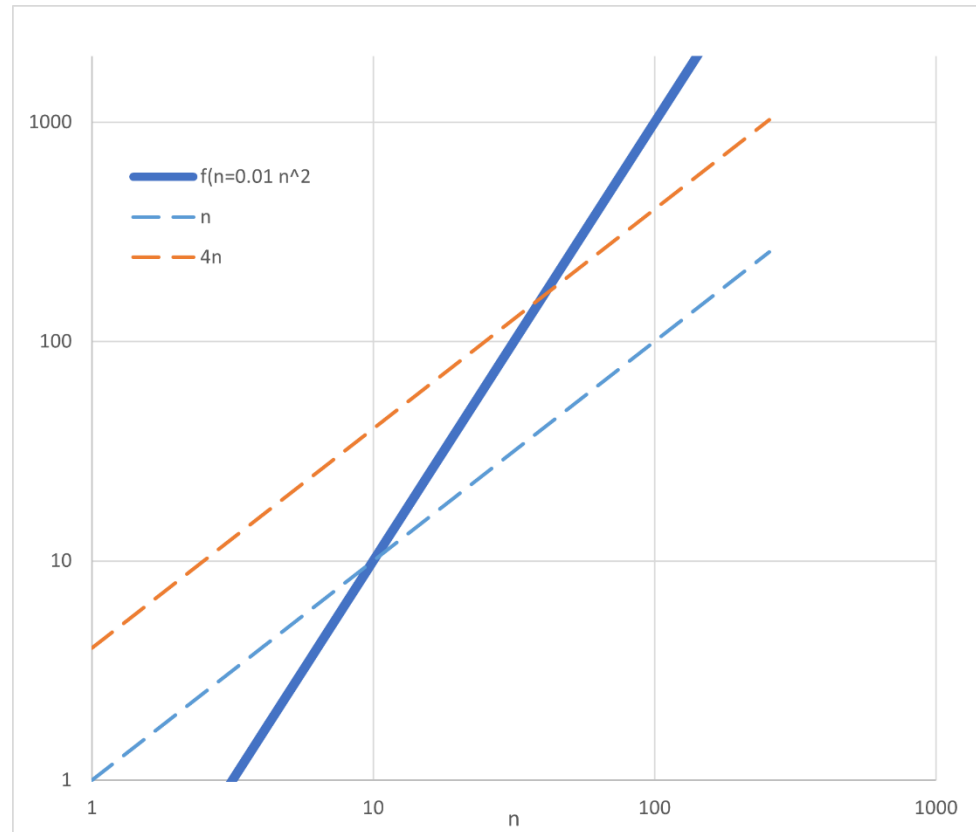
Big-Oh : higher powers win

$$\log(n^k) = k \log(n)$$

So higher powers
have larger slope and
will always win at
large enough n

n^{k1} is $O(n^{k2})$
if $k1 \leq k2$

It terms of $O(\cdot)$,
higher powers win
even if the
coefficients are small



Big Oh as a binary relation

- “f is $O(g)$ ” can be regarded as a binary relation between two functions f and g
 - Could be written: “ $O(f, g)$ ” or “ $f O g$ ”
- Generally, binary relations, R, are characterised by potential properties:
 - Reflexive: $x R x$
 - Symmetric: $x R y \text{ iff } y R x$
 - Transitive: $x R y \ \& \ y R z \rightarrow x R z$

Which of these are satisfied by big-Oh?

Big-Oh is Reflexive

- This is trivial:
- For any function

$f(n)$ is $O(f(n))$

as just take $c=1$, and use $\forall n. f(n) \leq f(n)$

Big Oh is not symmetric

- 1 is $O(n)$

But (from an earlier example)

- n is not $O(1)$

It only takes one counterexample to show that we cannot say it is symmetric

Big Oh is transitive

- Given “ f is $O(g)$ ” and “ g is $O(h)$ ”, then can we show “ f is $O(h)$ ” ?
 - E.g.
“ 1 is $O(n)$ ” and “ n is $O(n^2)$ ”
forces
“ 1 is $O(n^2)$ ” ?
- As usual, start by writing down what we know from the definitions:

Big Oh is transitive (2)

- Given “f is $O(g)$ ” and “g is $O(h)$ ” then can we show “f is $O(h)$ ” ?
- Exercise: Start by writing down what we know from the definitions:
exists c_1 n_1 c_2 n_2 s.t.
 $f(n) \leq c_1 g(n)$ for all $n \geq n_1$
 $g(n) \leq c_2 h(n)$ for all $n \geq n_2$
How do we get to “f is $O(h)$ ”?

Big Oh is transitive (3)

- Given “f is $O(g)$ ” and “g is $O(h)$ ” then can we show “f is $O(h)$ ” ?
 - exists c_1 n_1 c_2 n_2 s.t.
$$f(n) \leq c_1 g(n) \quad \text{for all } n \geq n_1$$
$$g(n) \leq c_2 h(n) \quad \text{for all } n \geq n_2$$
 - Mult 2nd inequality by c_1 (uses $c_1 > 0$)
$$c_1 g(n) \leq c_1 c_2 h(n) \quad \text{for all } n \geq n_2$$
 - Set $n_3 = \text{maximum}(n_1, n_2)$, and combine gives
$$f(n) \leq c_1 g(n) \leq c_1 c_2 h(n) \quad \text{for all } n \geq n_3$$
- Then can take $c = c_1 * c_2$ to complete the proof.

Big-Oh as a set

- Big Oh as a binary relation is reflexive and transitive but not symmetric
 - It behaves like " $\subseteq$ ", " $\in$ " or " $\leq$ ", not like " $=$ "
 - One might say $n \in O(n)$, and $2n+3 \in O(n)$, etc.
- So may help to think of " $O(n)$ " as a set of functions, with each function f in the set, $f \in O(n)$, satisfying " f is $O(n)$ ".
 - Or can say: $\{f\} \subseteq O(n)$
 - So, then $O(1) \subseteq O(n)$
 - Any function bounded above by a constant, is also bounded above by a linear function
 - This gives a closer match to hierarchical classification, similar to
Tom $\in$ Cats, Cats $\subseteq$ Mammals

Big-Oh as a set - notation

May help to think of " $O(n)$ " as a set of functions, with each function f in the set, $f \in O(n)$, satisfying " f is $O(n)$ ".

- Many texts use " $f = O(n)$ ", but
- I dislike this because usual intuition about "=" would make it natural to say:
 - $1 = O(1)$ and $1 = O(n)$ hence $O(1) = O(n)$ which is WRONG
 - E.g. $O(n)$ contains $f(n)=n$, but $O(1)$ does not
- Also, we cannot swap the order:
 - $1 = O(1)$ does not mean we can write " $O(1) = 1$ " which is meaningless

Big-Oh as a set (2)

- So may help to think of “ $O(n)$ ” as a set of functions, with each function f in the set, $f \in O(n)$, satisfying “ f is $O(n)$ ”.
- This allows to give sense to expressions such as

$$n^{O(1)}$$

to mean “ n to the power of $f(n)$ with $f(n)$ being $O(1)$ ”.

- Hence, this included n , n^2 , n^3 , etc
- Can sometimes be used as way to say “is some power law”
 - (Under “big-Oh as worst case of an algorithm” then $n^{O(1)}$ would make no sense!).

Summary & Expectations

- Know the definition of big-Oh well!
 - Understand why it is appropriate in CS
- Be able to apply it, and prove results on big-Oh of simple functions
- Big Oh as a binary relation is reflexive and transitive but not symmetric
 - It behaves like " $\subseteq$ ", " $\in$ " or " $\leq$ ", not like " $=$ "

Exercises (offline) (2)

Q1: Show $7n-2$ is $O(n)$

Q2: Show $3n^3 + 20n^2 + 5$ is $O(n^3)$

Q3: Show $3 \log n + 5$ is $O(\log n)$